

A gamified introduction to Python Programming

Lecture 13 Everything about strings

Matthieu DE MARI – Singapore University of Technology and Design



SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN



Outline (Chapter 13)

- What is a **string**? How are they related to **lists**?
- How to **index**, **slice** and **traverse** a string?
- What is the **immutability property** of strings?
- What are common **booleans on strings**?
- A complete description of **printing** and **formatting** with strings.
- How to **convert strings to lists** and vice-versa?
- What are **more advanced operations** on strings, such as splitting, stripping, character checking, sorting, etc.? (RTFM!)

From lists to strings

Important note: Lists and strings behave in a very similar way.

- **Lists:** multiple **elements** between **square brackets**, separated by commas
- **Strings:** multiple **characters** between **single or double quotes** (`'`, `"`), without any separators.

Strings are **roughly the same thing as a list of single characters**.

string = "Hello"

equivalent_list = ["H", "e", "l", "l", "o"]

Great news: This means that most functions, methods and operators working on lists and seen earlier will work on strings as well!

(Except for one minor difference, to be discussed later...)

Length of a string

Definition (**length** of a string):

The **length** of a string is simply defined as the number of characters in the string.

- This would be the same as the length of the equivalent list containing all single characters.
- This is obtained, as before via the **len()** function.
- **Spaces and special characters count as characters!**

```
1 a_string = "This is a string."  
2 print(a_string)  
3 print(len(a_string))
```

```
This is a string.  
17
```

string = "Hello"

equivalent_list = ["H", "e", "l", "l", "o"]

Using triple quotes and string formatting (\)

Problem: Need to use the single or double quote symbol in your strings?

1. Replace the single or double quotes at the beginning and end of the string **with three of these symbols** (""" or ```).
2. Use a **backslash symbol** (\) before a single or double quote in a string. This is called an **escape symbol** and is used in programming to indicate the character should be used “as a character”, without the special meaning it might have in the programming language.

You can also use some string formatting, such as:

- \n to start a **new line** (~ pressing the Enter key),
- and \t for **tabulating** (~ pressing the Tab key)

Using triple quotes and string formatting (\)

```
1  # A terrible computer science joke
2  joke1 = "Bob was heading to the supermarket."
3  joke2 = 'As he was leaving, his wife told him:'
4  joke3 = ''' "While you're out, pick up some eggs!" '''
5  joke4 = "He never came back. \"I'm really sad\", said the wife."
6  joke5 = "\nAlso, the store eventually ran out of eggs."
7  joke6 = "\t The End."
8  print(joke1)
9  print(joke2 + joke3)
10 print(joke4, joke5)
11 print(joke6, end = "(badum tsss!)")
```

Bob was heading to the supermarket.

As he was leaving, his wife told him: "While you're out, pick up some eggs!"

He never came back. "I'm really sad", said the wife.

Also, the store eventually ran out of eggs.

 The End.(badum tsss!)

The + and * operators on strings

As we have seen, the strings behave as lists.

- For this reason, the + operator will **concatenate two strings**,
- And the * operator will **repeat and concatenate** a string **n times**, assuming **n** is an **integer positive value**.

```
1 list1 = [1, 2, 3]
2 list2 = [4, 5, 6, 7]
3 n = 3
4 new_list = list1*n + list2
5 print(new_list)
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3, 4, 5, 6, 7]
```

```
1 str1 = "pika"
2 str2 = "chu"
3 n = 3
4 new_string = str1*n + str2
5 print(new_string)
```

```
pikapikapikachu
```

Indexing and slicing a string

As with lists, you can

- **index** a string, by using the bracket notation `[]` on a list. It returns the **character** at the index position.
- **slice** a string, by using the bracket notation `[]` on a list. It returns the **substring** with specified indexes.

As before, it helps to visualize the string as a list of characters.

```
1 # Calling/indexing a string as in lists
2 a_string = "This is a string."
3 print(a_string[0])
4 print(a_string[-2])
```

T
g

```
1 # Slicing a string is fine as well
2 a_string = "This is a string."
3 print(a_string[5:12])
4 print(a_string[-6:-1:2])
```

is a st
tig

string = "Hello"

equivalent_list = ["H", "e", "l", "l", "o"]

Immutable types

Definition (The **immutability** property of strings):

A major difference between lists and strings, however, is that strings are an **immutable** type.

- **Immutable** means that you can not modify some or parts of its elements/characters, by using the bracket notation, as you would with lists.
- Operation `s[i] = "c"` is not valid!

```
1 # Lists are mutable and can be updated
2 a_list = [1, 4, 7, 9]
3 a_list[2] = 11
4 print(a_list)
```

```
[1, 4, 11, 9]
```

```
1 # Strings are immutable
2 # (i.e. cannot be changed once defined)
3 a_string = "Your GPA is 2"
4 a_string[-1] = 5 # Does not work
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-3-736e16edf069> in <module>
      2 # (i.e. cannot be changed once defined)
      3 a_string = "Your GPA is 2"
----> 4 a_string[-1] = 5 # Does not work

TypeError: 'str' object does not support item assignment
```

```
1 # Strings are immutable
2 # (i.e. cannot be changed once defined)
3 a_string = "Your GPA is 2"
4 a_string[-1] = "5" # Stop it we said
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-4-a2c24ccda094> in <module>
      2 # (i.e. cannot be changed once defined)
      3 a_string = "Your GPA is 2"
----> 4 a_string[-1] = "5" # Stop it we said

TypeError: 'str' object does not support item assignment
```

Replacing parts of a string

Instead, the strings come with a **replace()** method.

- Its two parameter are **strings s1** and **s2** and the method is applied on a **third string s3**.
- We need to catch the output, which is a new string, consisting of the **third string s3**, where the substring **s1** has been replaced with **s2**.
- The **replace()** method replaces **all occurrences** of **s1** in **s3** with **s2**.

```
1 a_mistake = "British cuisine is the best."  
2 fixed = a_mistake.replace("British", "French")  
3 print(fixed)
```

French cuisine is the best.

Traversing a string

Because strings are roughly the same things as lists, they can be traversed using a **for** loop,

- Element-wise,

```
1  # Traversing a string
2  # (element-wise)
3  a_string = "Hello"
4  for character in a_string:
5      print("----")
6      print(character)
```

H

e

l

l

o

Traversing a string

Because strings are roughly the same things as lists, they can be traversed using a **for** loop,

- Element-wise,
- Index-wise,

```
1  # Traversing a string
2  # (index-wise)
3  a_string = "Hello"
4  for index in range(len(a_string)):
5      print("---")
6      print(a_string[index])
```

```
---
H
---
e
---
l
---
l
---
o
```

Traversing a string

Because strings are roughly the same things as lists, they can be traversed using a **for** loop,

- Element-wise,
- Index-wise,
- Index- and element-wise with **enumerate**.
- (Everything else, such as **zip**, nested loops, etc. also works!)

```
1  # Traversing a string
2  # (enumerate)
3  a_string = "Hello"
4  for index, character in enumerate(a_string):
5      print("---")
6      print(index, character)
```

```
---
0 H
---
1 e
---
2 l
---
3 l
---
4 o
```

The **in** keyword

Membership operator in strings:

As with lists, the **in** keyword can be used to check if a character or a substring appears in a string.

- It returns a Boolean set to **True** or **False** accordingly.
- Also works to check if a substring (not just a single character) appears.

```
1  # Membership keyword: in
2  a_string = "Pikachu"
3  bool1 = "k" in a_string
4  print("bool1: ", bool1)
5  bool2 = "chu" in a_string
6  print("bool2: ", bool2)
7  bool3 = "swag" in a_string
8  print("bool3: ", bool3)
```

```
bool1:  True
bool2:  True
bool3:  False
```

Activity 1 – Is it a palindrome

A **palindrome** is a word, number, phrase, or other sequence of characters which reads the same backward as forward, such as madam, racecar, etc.

- There are also numeric palindromes, including date/time stamps using short digits 11/11/11 11:11 and long digits 02/02/2020.
- Sentence-length palindromes may be written when allowances are made for adjustments to capital letters, punctuation, and word dividers, such as "A man, a plan, a canal, Panama!".

- Write a function **is_palindrome()**, which receives a **single word**, in a string variable **word** as its sole input parameter.
- It should return **True** if the string in question is a **palindrome**.
- And **False otherwise**.



KAYAK
TACO CAT
RACECAR
NEVER ODD OR EVEN
STEP ON NO PETS
UFO TOFU

Activity 2 – 1337 speak translators

Leet (or "1337") is a system of modified spellings used primarily on the Internet. It often uses character replacements in ways that play on the similarity of their glyphs via reflection or other resemblance.

Leet speak replaces the letters l with 1, the letters Z with 2, the letters E with 3, the letters A with 4, the letters S with 5, the letters T with 7, the letters B with 8, the letters O with 0.

For instance, the English sentence "HELLO THERE, HOW ARE YOU?" will write in leet speak as "H3LL0 7H3R3, HOW 4R3 YOU?".



It's tim3
t0 l34rn h0w
t0 r34d
4g41n

Activity 2 – 1337 speak translators

Leet (or "1337") is a system of modified spellings used primarily on the Internet. It often uses character replacements in ways that play on the similarity of their glyphs via reflection or other resemblance.

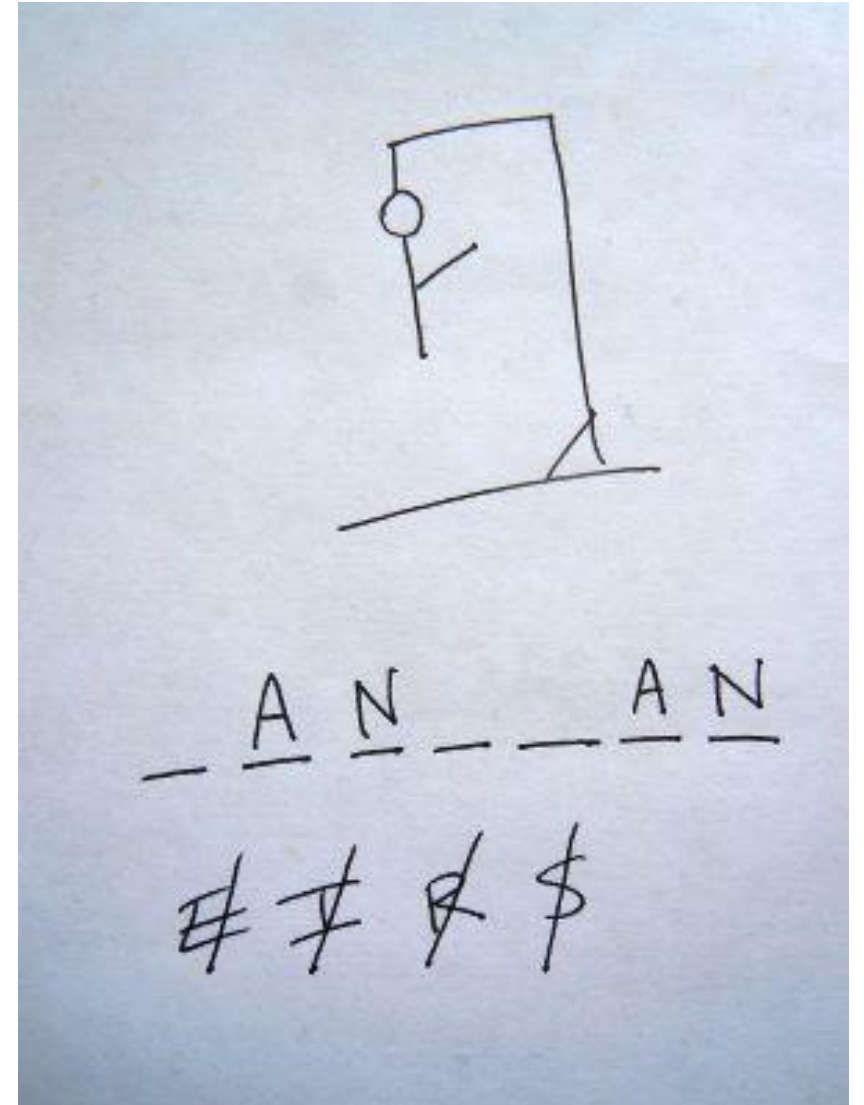
Leet speak replaces the letters I with 1, the letters Z with 2, the letters E with 3, the letters A with 4, the letters S with 5, the letters T with 7, the letters B with 8, the letters O with 0.

For instance, the English sentence "HELLO THERE, HOW ARE YOU?" will write in leet speak as "H3LL0 7H3R3, HOW 4R3 YOU?".

- Write a function **eng_to_leet()**.
- It receives a single parameter, **eng_str**, which corresponds to a sentence in English, written in capital letters.
- It returns the equivalent sentence in leet speak.
- Do so by modifying the letters with numbers, by following the rules we defined earlier.

Activity 3 – Hangman

The hangman is a game where one player thinks of a word, phrase or sentence and the other(s) tries to guess it by suggesting letters within a certain number of guesses.



Activity 3 – Hangman

Write a function **hangman()**, which receives four arguments:

- **hidden_word**: a hidden word that the player must guess. Normally, this word would be hidden from the player, but we will show it for now, as a way to check our function `hangman()` works as intended.
- **proposed_letter**: a single letter string, corresponding to a letter, which the player just proposed.
- **letters_list**: a list of letters, containing the previous propositions made by the user.

This function returns three arguments:

- **revealed_word**: The revealed word consists of the `hidden_word`, where the letters in the parameters `proposed_letter` and `letters_list` are shown. the other letters are replaced with star (*) characters.

For instance, if the parameters are

- `hidden_word = "bulbasaur"`,
- `proposed_letter = "u"`,
- and `letters_list = ["e", "a", "r"]` are passed to the function `hangman()`,

Then it will return `revealed_word = "*u**a*aur"`.
Note that all occurrences of the proposed letters should be displayed.

Boolean operations on strings

Boolean operations on strings:

- As you have seen earlier, we can use `==` (resp. `!=`) to check if two strings are identical (resp. different).

The comparisons also work on strings but with a different behavior.

- The comparison operators (`>`, `<`, `>=`, `<=`) can also be used.
- These comparison operators compare the alphabetical order between strings.

```
1 string1 = "Matt"
2 string2 = "Chris"
3 string3 = "Oka"
4 string4 = "Matt"
5 bool1 = (string1 == string2)
6 print("bool1: ", bool1)
7 bool2 = (string1 == string4)
8 print("bool2: ", bool2)
9 bool3 = (string1 > string2)
10 print("bool3: ", bool3)
11 bool4 = (string1 <= string3)
12 print("bool4: ", bool4)
```

```
bool1: False
bool2: True
bool3: True
bool4: True
```

Strings formatting with `format()`

We have seen how the **`format()`** method could be used to insert the values of some variables inside a string.

- Turns out it can operate with indexes or even keywords.
- Some arguments can also be passed to the placeholders {}, for more advanced formatting, but this is out of the scope of this class.
- <https://docs.python.org/3/library/string.html#formatstrings>

```
1  # Minimal format
2  print("Your {} is now level {}. Well done {}!".format("Pikachu", 15, "Pikachu"))
3  # Indexed placeholders
4  print("Your {1} is now level {0}. Well done {1}!".format(15, "Pikachu"))
5  # Keyword arguments
6  print("Your {name} is now level {level}. Well done {name}!".format(name = "Pikachu", level = 15))
```

```
Your Pikachu is now level 15. Well done Pikachu!
Your Pikachu is now level 15. Well done Pikachu!
Your Pikachu is now level 15. Well done Pikachu!
```

A quick word on F-strings

A quick word on **F-Strings**:

- The latest versions of Python (3.7+) came with **F-strings**: the **F-strings** are strings, but they are preceded by the letter **F**, to indicate that it contains some placeholders **{}** to be formatted.
- In F-strings, you can directly pass some variable names **inside** the placeholders, which is simpler than using **format()**.

```
1  # F-strings
2  name = "Pikachu"
3  level = 15
4  fstring = F"Your {name} is now level {level}."
5  print(fstring)
6  print(type(fstring))
```

```
Your Pikachu is now level 15.
<class 'str'>
```

Lists $\leftrightarrow$ Strings conversions

Because lists and strings are roughly the same thing, it is sometimes convenient to convert one into the other.

- It is sometimes more useful to convert one into the other to **get rid of the immutability property of strings!**
- **Str $\rightarrow$ List:** Easily done with **list()**, which gives a list of characters.

```
1  # String to list conversion
2  so_noisy = "PIKAPI!"
3  print(so_noisy)
4  noise_as_list = list(so_noisy)
5  print(noise_as_list)
```

```
PIKAPI!
```

```
['P', 'I', 'K', 'A', 'P', 'I', '!']
```

Lists $\leftrightarrow$ Strings conversions

The converse operations is somewhat more difficult.

- **List $\rightarrow$ Str:** done with the **join()** method, often applied on an empty string **""**.
- Applying it on something else than an empty string leads to... interesting results.

```
1  # List of strings joining
2  print(noise_as_list)
3  noise_as_string = ''.join(noise_as_list)
4  print(noise_as_string)
5  # List of strings joining
6  HELL_NO = "PIKA".join(noise_as_list)
7  print(HELL_NO)
```

```
['P', 'I', 'K', 'A', 'P', 'I', '!']
```

```
PIKAPI!
```

```
PPIKAIPKAKPIKAAPIKAPPIKAIPKA!
```


Splitting a string into a list of strings

If needed, you can **split a large string into a list of smaller ones**, with the **split()** method.

- By default, it returns a list of the different words in the string, **using whitespaces as separators**.
- You can also specify a substring to use as a separator instead of whitespaces.
- Convenient for dates (14/10/2020), times (14:52:26), etc.

```
1 # Splitting a string
2 so_annoying = "PIKA PIKA PIKA? PIKAPI PIKA PIKA PI!"
3 print(so_annoying)
4 print(so_annoying.split())
5 # Splitting a string
6 my_birthday = "02-Mar-1989"
7 print(my_birthday)
8 print(my_birthday.split("-"))
```

```
PIKA PIKA PIKA? PIKAPI PIKA PIKA PI!
['PIKA', 'PIKA', 'PIKA?', 'PIKAPI', 'PIKA', 'PIKA', 'PI!']
02-Mar-1989
['02', 'Mar', '1989']
```

Characters types check in strings

You can check for characters types in a string with 3 methods.

- **isalpha()** returns **True** if characters are letters only; and **False** otherwise.
- **isdigit()** returns **True** if the characters are digits only; and **False** otherwise.
- **isalnum()** returns **True** if the characters are letters and digits only; and **False** otherwise.

```
1 # The isalpha() method
2 my_first_name = "Matt"
3 my_last_name = "De Mari"
4 print(my_first_name.isalpha())
5 print(my_last_name.isalpha())
```

True
False

```
1 # The isdigit() method
2 my_SG_phone = "00000000"
3 my_FR_phone = "+33 0000000000"
4 print(my_SG_phone.isdigit())
5 print(my_FR_phone.isdigit())
```

True
False

```
1 # The isalnum() method
2 my_age = "31years" # Sigh
3 my_email = "best_teacher_in_SUTD@sutd.edu.sg"
4 print(my_age.isalnum())
5 print(my_email.isalnum())
```

True
False

Activity 4 - Create password from sentence

A **passphrase** is a sentence which can be used to create passwords that are difficult to crack and easy to memorize.

- The user simply needs to create a memorable sentence containing multiple words.
- Later on, the user will assemble a password by alternating the first letter of each word in the passphrase and the number of characters in it. In addition, add the special characters when they appear.

For instance, the sentence

**"I used to be an adventurer like you until
I took an arrow to the knee"**

can be used to generate the following password

"I1u4t2b2a2a10l4y3u5l1t4a2a5t2t3k4"

Write a function **generate_password()**, which receives a passphrase as its only parameter, and returns a single output, consisting of a string generated according to the method we described.

Activity 5 – Password checker

Write a function **password_check()**, which receives a single parameter, password, consisting of a string variable.

This function returns a boolean with value True if:

- the password has at least 8 characters,
- and the password contains at least one letter
- and the password contains at least one number,
- and the password contains at least one special character (i.e. a non-number, non-letter character).

If any of these conditions is not met, the password is deemed invalid (not strong enough) and the function returns False.

As usual, RTFM!

The string objects in Python have **many more built-in methods and functions** you could use.

- We **do not expect you to know** about all these methods and functions, nor about how they work in detail.
- Just know where to find the information if you need it.
- And do not be afraid to **read the Python documentation!**

String type documentation (**RTFM!**)

<https://docs.python.org/2/library/string.html>



The `find()` method

The strings come with an additional method, called `find()`, which looks for a character or substring inside another string.

- It returns the index of the first appears of said character or substring inside the string.
- If the character or substring does not appear, it returns -1.

```
1  # Find method in strings
2  a_string = "Pikapikapikachu"
3  print(a_string.find("k"))
4  print(a_string.find("p"))
5  print(a_string.find("chu"))
6  print(a_string.find("swag"))
```

```
2
4
12
-1
```

Sorting strings and lists of strings

We have seen earlier how the **sorted()** function could be used to sort a list of strings alphabetically.

- It can also be applied on a single string. This produces a list of strings, this time reorganized in alphabetical order.

```
1  # Sorting strings and lists of strings
2  string1 = "BDEFGAC"
3  print(sorted(string1))
4  strings_list = ["Matt", "Chris", "Oka"]
5  print(sorted(strings_list))
```

```
['A', 'B', 'C', 'D', 'E', 'F', 'G']
['Chris', 'Matt', 'Oka']
```

Additional: functions, strip-cleaning a string

Strip-cleaning a string removes the extra spaces, if any, at the beginning and end of a string.

- If a substring is passed to the **split()** method, it will remove the occurrences of these extra characters at the beginning and end of the string.

```
1  # Stripping a string
2  the_ISS = "        This string is floating in spaces        "
3  print(the_ISS)
4  print(the_ISS.strip())
5  # Stripping a string
6  allergic_to_stars = "***COUGH**COUGH*****"
7  print(allergic_to_stars)
8  print(allergic_to_stars.strip("*"))
```

```
        This string is floating in spaces
This string is floating in spaces
***COUGH**COUGH*****
COUGH**COUGH
```


Once more for those in the back

The string objects in Python have **many more built-in methods and functions** you could use.

- We **do not expect you to know** about all these methods and functions, nor about how they work in detail.
- Just know where to find the information if you need it.
- And do not be afraid to **read the Python documentation!**

String type documentation (**RTFM!**)

<https://docs.python.org/2/library/string.html>





Conclusion (Chapter 13)

- What is a **string**? How are they related to **lists**?
- How to **index**, **slice** and **traverse** a string?
- What is the **immutability property** of strings?
- What are common **booleans on strings**?
- A complete description of **printing** and **formatting** with strings.
- How to **convert strings to lists** and vice-versa?
- What are **more advanced operations** on strings, such as splitting, stripping, character checking, sorting, etc.? (RTFM!)

slido

Please download and install the Slido app on all computers you use



What is the output produced by this code?

① Start presenting to display the poll results on this slide.

slido

Please download and install the Slido app on all computers you use



What is the output of this code?

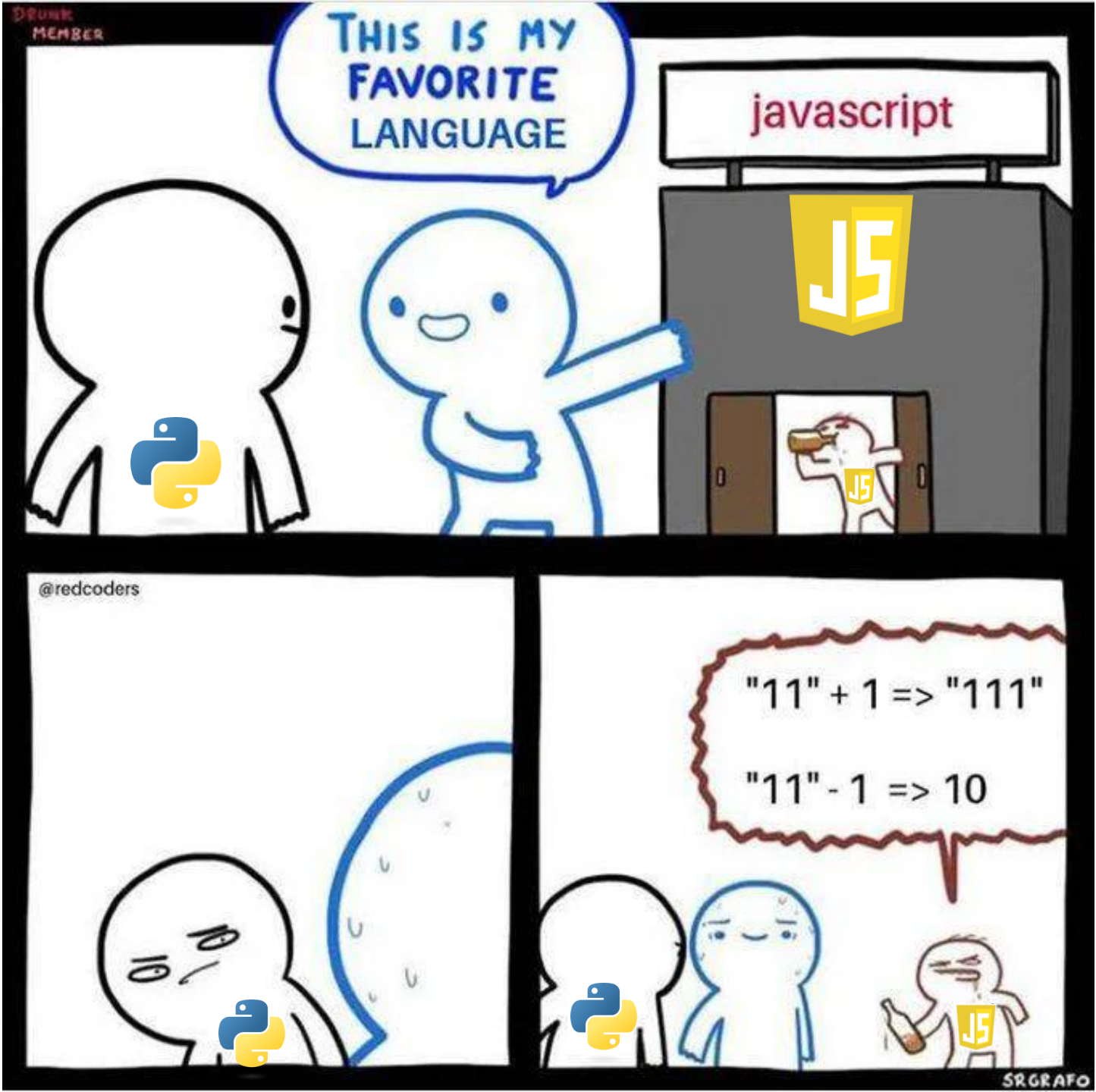
① Start presenting to display the poll results on this slide.

**SEES YOU ARE
TRYING TO ADD/SUBTRACT
A STRING AND AN INT**



**REFUSES TO OPERATE
AND GIVES YOU
A CLEAN ERROR MESSAGE**

Meanwhile...





Which slicing should we use on line 2 to produce a True on line 3?

① Start presenting to display the poll results on this slide.

slido

Please download and install the Slido app on all computers you use



What will be printed?

① Start presenting to display the poll results on this slide.



**Given a string $s =$
"MatthieuDeMari", which of
these will produce a True?**

① Start presenting to display the poll results on this slide.